

Project Name : Gopher Industries Food Remedy API

Student Name : M. Jayadhwaj Reddy

Student ID : 225678253

Ticket No. : RE017

RE017 – Firebase/Cloud Usage and Offline Feasibility Assessment

1. Introduction

The purpose of this assessment is to understand how much the Food Remedy application currently depends on Firebase and other cloud services, and to explore whether some parts of the application could still work when a user has limited or no internet access.

The review focuses on Firestore usage, SQLite-based local storage, synchronisation between cloud and local data, caching behaviour, and the practical differences between cloud-only, offline-only, and hybrid approaches.

The current Food Remedy documentation shows that Firestore is used as the main backend data source for several features, including cart information, product retrieval, and meal-plan generation. This means that the application currently relies heavily on cloud connectivity for a number of important functions.

At the same time, the repository also includes local SQLite storage and synchronisation logic, suggesting that the application already has the beginnings of a hybrid cloud/offline architecture.

2. Current Firebase and Cloud Usage

Food Remedy currently uses Firebase and Firestore for several core application functions. The backend API documentation identifies Firestore as the primary source of data. User cart records are stored under paths such as `users/{userId}/cart/{productId}`, while individual products are retrieved from the `PRODUCTS/{barcode}` collection. Meal-plan generation also requires product information to be loaded from Firestore before it is processed by the classification logic.

The database pipeline also relies on Firestore for seeding processed product information. Existing documentation describes schema validation, batched writes, retry behaviour, and controls around Firestore operations.

This level of dependency means that some functions cannot simply be moved offline without affecting the way the application currently works.

Firebase is designed to support applications that combine cloud and offline behaviour. Cloud Firestore can cache previously accessed information and allow local reads and writes while the device is offline. When connectivity returns, the client can synchronise local changes back to Firestore [1].

3. Existing Local and Offline Support

One useful finding from the repository is that Food Remedy is not completely cloud-only. The profile synchronisation service already contains functions for reading profiles from SQLite, saving them locally, synchronising profiles to Firebase, and bringing cloud data back into SQLite. The related test documentation also refers to duplicate prevention, conflict handling, timestamps, and maintaining data consistency.

This is important because it shows that the project already has practical experience with a hybrid model.

The architecture documentation also describes a product scan flow where the application can retrieve a product from Firestore or use a local cache. When the application is offline, it can display the last-known risk information and prompt the user to verify it again once connectivity is restored.

This type of behaviour is consistent with Firebase's offline model, where cached information can be available without a network connection, but cached data may be stale or incomplete [1].

4. Feature-by-Feature Feasibility Assessment

Feature	Current Dependency	Offline Feasibility	Recommended Approach
Authentication	Firebase Authentication	Limited offline capability	Keep mainly cloud-based, while allowing an existing signed-in session to persist
User profiles	Firebase + SQLite	High	Continue using hybrid synchronisation
Product lookup	Firestore + local cache	Moderate to high	Keep Firestore authoritative but cache recent products locally
Barcode scan history	Local data is practical	High	Store locally and optionally sync
Cart	Firestore	Moderate	Allow local changes and sync them once online
User preferences	SQLite/local storage	High	Store locally and sync when needed
Risk badges	Cached product information	Moderate	Allow cached display with a clear warning
Meal-plan generation	Firestore + classification logic	Low	Keep mainly cloud-based
Recommendations	Current product/profile information	Moderate	Cache previous results but refresh them online
Product catalogue	Firestore	Low for full offline copy	Keep central catalogue in cloud and cache selected products
Firestore seeding	Firestore backend operation	None	Keep cloud-based
Analytics/monitoring	Cloud-dependent	Low	Keep cloud-based

Firebase Authentication can preserve the current user's authentication state between app sessions, and the JavaScript SDK supports persistent authentication state. However, activities that require new server-side verification still depend on connectivity [2], [3].

5. Cloud, Local and Hybrid Comparison

Area	Cloud-Based	Local/Offline	Hybrid
Cost	Ongoing cloud usage charges	Lower cloud usage	Balanced approach
Performance	Depends on network speed	Fast local access	Fast local access with cloud refresh
Reliability	Internet connection required	Works without internet	Can continue offline and sync later
Data freshness	Usually current	May become outdated	Cached locally and refreshed online
Complexity	Easier central management	Simple for isolated device data	More complex due to synchronisation
Cross-device access	Strong	Limited	Good after synchronisation
Security	Centralised access	Device storage needs	Requires security in both

	controls	protection	places
Conflict handling	Less common	Usually limited	Must be planned carefully
Scalability	Strong	Device-dependent	Strong when designed properly

The hybrid model adds some technical complexity, but it provides the most balanced outcome for Food Remedy. Users can continue accessing selected information during connectivity problems, while Firestore remains the source of truth for current and shared data.

6. Local Storage Options

SQLite is already present in Food Remedy, so it is the most practical choice for structured offline information. It could be used for:

- user profiles
- scan history
- recently viewed products
- cached product details
- preferences
- pending offline changes

Using the technology that is already part of the project avoids unnecessary redevelopment and keeps the architecture more consistent.

For smaller values such as simple settings, feature flags, or session-related values, AsyncStorage could also be suitable. Firebase Authentication itself supports React Native persistence through AsyncStorage [2]. However, SQLite remains more appropriate for structured application data because it supports organised tables and queries.

Introducing another local database such as Realm or MMKV would add extra complexity without a clear need at this stage.

7. Synchronisation Risks

7.1 Stale Data

Cached product information can become outdated while the device is offline. For example, ingredients, nutritional values, or allergen information may change in the cloud database. Firestore documentation notes that locally cached results may be stale or incomplete when the application is offline [1]. Food Remedy should therefore store timestamps and make it clear when users are viewing previously cached information.

7.2 Conflicting Updates

A user could change the same profile or cart record on more than one device. The repository already contains profile synchronisation logic that considers duplicate prevention, timestamps, conflict handling, and consistency. This approach could be extended to other offline-capable features. Firestore itself uses a last-write-wins approach when multiple changes are made to the same document while offline [1].

7.3 Failed Synchronisation

A locally saved change may fail when the device reconnects. Instead of discarding the change, Food Remedy should keep it in a queue and retry later. The application should also be able to indicate whether information is fully synchronised.

7.4 Duplicate Data

Repeated offline actions could create duplicate entries when synchronisation happens later. Stable IDs and duplicate checking should therefore be used before writing information back to Firestore.

7.5 Security

Local storage creates a different type of security risk because information is stored directly on the user's device. Only information required for offline functionality should be stored locally. Sensitive information should not be placed in unsecured local storage, and authentication state should use appropriate persistence mechanisms. Firebase supports different authentication persistence options depending on whether the session should survive application restarts or only remain available temporarily [3].

8. Reliability and User Experience

Offline functionality would be particularly useful when users are shopping in places with weak mobile reception. For example, a user who previously scanned a product could still be able to view the saved product information without waiting for the application to reconnect.

However, Food Remedy should avoid making old information look current. A message such as "Showing saved product information. Connect to the internet to verify the latest details." would make the limitation clear to users.

This is especially important because Food Remedy deals with nutritional and allergen information, where outdated information could affect a user's decision. The purpose of offline support should therefore be to improve continuity and convenience rather than replace the cloud completely.

9. Recommendations for the Backend and Database Teams

1. Keep Firestore as the main source of truth for shared product and account information.
2. Continue using SQLite for local profiles and extend it to appropriate cached information.
3. Build on the existing hybrid model rather than attempting a completely offline application.
4. Cache recently scanned and recently viewed products locally.
5. Allow selected actions, such as profile or cart changes, to be queued when offline and synchronised later.
6. Store timestamps such as lastUpdated and syncedAt so the application can identify stale records.
7. Apply clear conflict-resolution rules whenever local and cloud records differ.
8. Add retry mechanisms for failed synchronisation.
9. Avoid storing unnecessary sensitive user information locally.
10. Clearly inform users when information has been loaded from local cache rather than the current cloud database.
11. Keep authentication, Firestore seeding, central product storage, and other shared backend operations mainly cloud-based.
12. Improve automated testing around SQLite and synchronisation services, because existing project documentation identifies limited dedicated coverage in these areas.

10. Recommended Architecture Direction

Based on the current architecture, a hybrid cloud and local model is the most practical option for Food Remedy. Moving everything to the cloud would mean that several useful parts of the application could stop working whenever connectivity is poor. At the opposite extreme, moving the application fully offline would introduce problems with authentication, current product information, cross-device access, database updates, and consistency.

The hybrid approach provides a middle ground.

Cloud services should continue handling:

- authentication and account validation
- central product information
- shared user data
- Firestore seeding
- meal-plan generation
- recommendations that require current data
- analytics and monitoring

Local storage can support:

- user profiles
- scan history
- recently viewed products
- cached product information
- preferences
- pending offline changes

This direction also fits the existing Food Remedy implementation because Firebase and SQLite are already being used together for profile synchronisation.

11. Conclusion

The assessment indicates that Food Remedy does not need to become either completely cloud-based or completely offline. The current project already contains elements of a hybrid architecture, including Firestore, Firebase, SQLite, local caching, and cloud/local profile synchronisation.

Some features should remain cloud-based because they depend on current shared information. These include authentication, central product data, Firestore seeding, and server-dependent meal-plan functionality. Other functions, such as user profiles, scan history, preferences, and previously accessed product information, are good candidates for offline support.

For this reason, the recommended direction is to continue developing the hybrid model. Firestore should remain the authoritative source, while SQLite and local caching should provide limited offline access and improve reliability when connectivity is poor.

The main areas that would need attention as offline functionality expands are synchronisation, conflict handling, stale data, failed updates, local security, and clear communication to users when cached information is being displayed.

References

- [1] Google, "Access data offline," Firebase Documentation. [Online]. Available: <https://firebase.google.com/docs/firestore/manage-data/enable-offline>.
- [2] Google, "Firebase Authentication JavaScript API reference," Firebase Documentation. [Online]. Available: <https://firebase.google.com/docs/reference/js/auth>.
- [3] Google, "Authentication State Persistence," Firebase Documentation. [Online]. Available: <https://firebase.google.com/docs/auth/web/auth-state-persistence>.
- [4] Food Remedy API Team, "API Documentation," Food Remedy API Repository, 2026.
- [5] Food Remedy API Team, "BE001 Test Coverage Baseline," Food Remedy API Repository, 2026.
- [6] Food Remedy API Team, "Data Architecture Overview," Food Remedy API Repository, 2025.
- [7] Food Remedy API Team, "DB028 Pipeline Reliability and Seeding Documentation," Food Remedy API Repository, 2025.